IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 23 June 2025, accepted 17 July 2025, date of publication 23 July 2025, date of current version 1 August 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3592030

RESEARCH ARTICLE

PEARL: An Adaptive and Explainable Hardware Trojan Detection Using Open Source and Enterprise Large Language Models

RIPAN KUMAR KUNDU[1], (Student Member, IEEE), KHURRAM KHALIL[1], (Student Member, IEEE), ERIC GARCIA[2], ETHAN GRASSIA[3], PRASAD CALYAM[1], (Senior Member, IEEE), AND KHAZA ANUARUL HOQUE[1], (Senior Member, IEEE)

[1]Department of Electrical Engineering and Computer Science, University of Missouri, Columbia, MO 65201, USA
[2]Columbia College, Columbia, MO 65201, USA
[3]Department of Computer Science, Loyola University, Chicago, IL 60660, USA

Corresponding author: Khaza Anuarul Hoque (hoquek@missouri.edu)

ABSTRACT The Integrated Circuit (IC) supply chain risk allows attackers to implant hardware Trojans (HT) in various stages of chip production. To counter this, different machine learning (ML) and deep learning (DL)-based methods have been developed to detect HTs. However, these methods require massive amounts of high-quality labeled data for effective training, extended training times for accurate HT detection, limited generalization to novel or unseen HTs, and insufficient capability to explain the detected HTs. Recent studies have started exploring the potential of Large language models (LLMs) for hardware security tasks. However, there are no current studies that explore, study, and compare the applicability of open-source vs. enterprise LLMs for efficient HT detection and explanation of the detected HT. To close the gap, we propose an innovative HT detection and explanation method by leveraging the knowledge of a pre-trained LLM, namely enterprise application programming interface (API)-enabled a Generative Pre-trained Transformer (GPT)-3.5 Turbo, Google Gemini 1.5 Pro and open-source LLMs: Meta AI Llama-3.1 and DeepSeek AI DeepSeek-V2 models, which have already been trained on massive and diverse datasets and is capable of providing the reasoning of the detected HT. Specifically, we apply In-Context Learning (ICL)-based mechanisms: zero-shot, one-shot, and few-shot learning strategies (e.g., register transfer level (RTL) files (Verilog) of the circuit) to adopt this model for HT detection and explanation tasks. We validate our proposed approach on diverse circuit design benchmarks from Trust-Hub and ISCAS (85 and 89). Our experimental results show that the proposed few-shot learning-based enterprise-API-enabled GPT-3.5 Turbo and open-source DeepSeek-V2 LLM models detect unknown HTs with an accuracy of 97% and 91% and drastically reduce the training time compared to state-of-the-art techniques. Furthermore, after detecting the HT, they provide human-centric reasoning/explanation, reinforcing transparency and trust in the IC supply chain through its understanding.

INDEX TERMS Hardware trojan, deep learning, large language model, in-context learning, explainable AI.

I. INTRODUCTION

With the increasing demand for integrated circuits (ICs), most semiconductor manufacturing companies rely on the global supply chain, which helps reduce design costs and meet time-to-market deadlines. However, due to this globalization and untrusted third-party vendors in the IC supply chain, Hardware Trojan (HT) insertion has emerged as a significant concern in today's fabless semiconductor manufacturing since adversaries can insert malicious modifications for

The associate editor coordinating the review of this manuscript and approving it for publication was Christian Pilato.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

various reasons, such as information leakage, erroneous execution, or performance degradation on the chip [1], [2]. Recently, machine learning (ML)/deep learning (DL)-based data-driven methods have emerged as powerful tools for detecting HTs [2], [3], [4], [5], [6], [7]. Despite the great success of these ML/DL models in HT detection tasks, these methods have a few limitations: (i) state-of-the-art (SOTA) ML/DL-based HT detection models often require higher computational resources and extended training times due to training the model from scratch to achieve accurate prediction [1]. For instance, training a DL model to build an effective HT detection scheme involves iteratively adjusting numerous parameters within complex neural network architectures, necessitating significant computational resources and time-intensive optimization processes; (ii) these models require massive amounts of high-quality label data for effective training. However, acquiring massive amounts of high-quality label data in HT detection schemes for training such models is challenging due to the unknown nature of the Trojan types [2], [5], and (iii) these models often lack transferability and human-centric explainability, limiting their applicability to familiar environments and impeding their performance in unfamiliar environments (e.g., unknown benchmarks and unknown Trojan types). Moreover, it requires applying different post-hoc explanation techniques (e.g., SHAP) [8] to explain the ML/DL-models decision regarding the detected HT, which does not provide a human-centric explanation or actual reasoning behind the detected HT.

Limited research has been conducted to address these above-mentioned research gaps. Pan et al. [1] used zero-shot learning (ZSL)-based pre-trained graph convolutional network (GCN) and metric learning-based methods to detect HT. However, their proposed method requires extensive labeled data for accurate training to build a pre-trained model, limited generalization to novel or unseen HTs, and is unable to provide an explanation/reasoning of the detected HT. Recently, pre-trained large language models (LLMs) have been significantly driven by the rapid development of natural language processing (NLP) [9], computer vision (CV) [10], etc., Moreover, these models exhibit remarkable performance on tasks they were trained for and quickly adapt to other novel and complex tasks. This is made possible through an In-Context Learning (ICL) mechanism, which allows these models to learn from a limited number of inputs, commonly referred to as few-shot or one-shot prompts, even under zero-shot conditions (without training and fine-tuning), and later generate rationales to provide sophisticated reasoning or explanation on the given tasks (e.g., HT detection). Recent studies have started exploring the potential of LLMs for hardware security tasks. For instance, Chaudhuri et al. [11] used LLMs to identify, localize, and iteratively correct analog Trojans in A/MS circuits. In contrast, Hayashi et al. [12] applied a combination of open-source LLMs- and ML-based methods for HT detection tasks. While authors in [13] used pre-trained LLMs in a zero-shot setting for HT detection. However, their works have several limitations. Notably, their methods do not utilize the rich semantic knowledge embedded in LLMs to generate human-centric explanations, that is, to articulate *why a specific trigger or payload was classified as malicious?* This lack of interpretability hinders the trust, auditability, and usability of the detection system for hardware designers, verification engineers, and security auditors. Furthermore, these works used LLMs in a purely zero-shot fashion, without domain-specific fine-tuning or in-context learning (ICL), which raises concerns about robustness when encountering novel or obfuscated Trojan variants. *Furthermore, there are no current studies that explore, study, and compare the applicability of open-source vs. enterprise LLMs for efficient HT detection and explanation of the detected HT.* Thus, a significant research gap remains in exploring and enhancing the application of LLMs for efficient HT detection and explanation tasks.

This paper proposes an innovative HT detection and explanation method. The main contributions of this paper are as follows:

- **Innovative and Efficient HT Detection:** We propose a novel HT detection and explanation method by leveraging the knowledge of an LLM model without training a model from scratch. Specifically, we adopt four pre-trained LLM models, namely enterprise application programming interface (API)-enabled Generative Pre-trained Transformer ((GPT)-3.5 Turbo model [10] and Google Gemini 1.5 Pro [14] and open-source LLM Meta AI (Llama)-3.1 [15] and DeepSeek-V2 [16] LLM models for the HT detection and explanation task, which has already been trained on massive and diverse datasets (e.g., texts, codes, etc.,) and is capable of providing the reasoning of the detected HT. Our approach employs In-Context Learning (ICL) mechanisms: ZSL (without fine-tuning LLM models), few-shot, and one-shot learning (fine-tuning LLM models) for HT detection and explanation tasks. Specifically, it performs Trojan detection in unknown scenarios (e.g., complex benchmarks with unknown HTs).
- **Data Integration:** The proposed method leverages different data, such as register transfer level (RTL) files (Verilog) of the circuit, to enhance both the accuracy and comprehensiveness of HT detection. This integration allows the model to effectively address complex benchmarks with unknown Trojans, improving detection performance.
- **Human-Centric Reasoning and Explanation:** After HT detection, the model provides human-centric reasoning and explanations by incorporating human-annotated rationales, such as chain-of-thought prompting. This feature enhances the transparency of the detection process, offering insights into the influence of each input feature on the model's predictions.
- **Extensive Validation and Golden-Free Solution:** We validate our proposed approach using diverse benchmarks from Trust-Hub [17] and ISCAS (both ISCAS'85

and ISCAS'89) [18], demonstrating its robustness and applicability. Furthermore, our method is a golden-free solution that does not require any golden model during HT detection and can be applied to real-life examples without golden models. Our extensive experimental evaluation demonstrates that our proposed method has several advantages compared to SOTA techniques. For instance, our experimental results demonstrate higher HT detection accuracy (97% for enterprise-API-enabled GPT-3.5 Turbo and 91% for open-source DeepSeek-V2 LLM models in FSL setting) for unknown Trojans and unseen benchmarks, which outperforms SOTA HT detection methods in terms of detection accuracy [1], [5], [6], [7]. Furthermore, our results show that a few-shot learning-based fine-tuned LLM models (i.e., the enterprise-API-enabled GPT-3.5 Turbo and Gemini-1.5 Pro LLM models) drastically reduced the training time in the HT detection tasks compared to the SOTA works (training model from scratch) [1], [5], [6], [7].

To the best of our knowledge, this is the first work that demonstrates the use of the enterprise API-enabled GPT-3.5 Turbo and Google Gemini 1.5 Pro, and the open-source Llama 3.1 and DeepSeek V2 models for HT detection and explanation tasks.

TABLE 1. Comparison of our proposed method with state-of-the-art (SOTA) related work in ML/DL-based model along with LLM-based method in HT detection and explanation tasks, where *WT LLM* denotes without LLM which require an ML/DL model from scratch for HT detection tasks, *LLM-EP* denotes enterprise API-based pre-trained LLMs, *LLM-OP* denotes open-source pre-trained LLMs, and *TR-EX* denotes an explanation of the detected Trojans.

Prior works	WT LLM	LLM-EP	LLM-OP	TR-EX
Salmani et al. [7]	Yes	No	No	No
Hasegawa et al. [5]	Yes	No	No	No
Yasaei et al. [29]	Yes	No	No	No
Faezi et al. [6]	Yes	No	No	No
Gubbi et al. [2]	Yes	No	No	No
Pan et al. [8]	Yes	No	No	Yes
Vishwakarma et al. [30]	Yes	No	No	Yes
Jiang et al. [31]	Yes	No	No	No
Vishwakarma et al. [32]	Yes	No	No	No
Pan et al. [1]	Yes	No	No	No
Faruque et al. [33]	No	Yes	Yes	No
Wan et al. [34]	No	Yes	Yes	No
Faruque et al. [13]	No	Yes	Yes	No
Chaudhuri et al. [11]	Yes	Yes	No	No
Hayashi et al. [12]	No	Yes	Yes	No
Our Proposed	No	Yes	Yes	Yes

II. BACKGROUND AND RELATED WORK

This section briefly discusses and reviews the current understanding of HT detection and several key aspects pertinent to our work: traditional hardware Trojan detection, ML/DL-based HTs detection, and large language models in HT detection tasks.

A. TRADITIONAL HARDWARE TROJANS DETECTION

Traditional HT detection techniques can be broadly categorized into three primary approaches: formal verification (FV) [19], [20], [21], logic testing (LT) [22], [23], [24], [25], [26], [27], and side-channel analysis (SCA) [8], [26], [28]. Formal verification involves converting the hardware design netlist into a proof-checking format to verify whether the intellectual property (IP) complies with predefined security specifications. For instance, Zhao et.al. [20] proposed a formal verification-based framework that combined GLIFT and bounded model checking to enable the static verification of information flow within hardware systems and detecting malicious information flows on those systems. Although formal verification is conceptually simple, its detection capabilities are constrained by the limited coverage of the predefined properties, which may not account for all potential malicious modifications. On the other hand, Logic testing methods, such as statistical test generation [22], focus on triggering Trojans by creating specific input patterns. For instance, authors in [22] proposed a test pattern generation method based on multiple excitations of rare logic conditions at internal nodes. While Nourian et. al. [24] used an advised genetic algorithm that creates effective test vectors for HT detection tasks (details will be found in this survey [25]). However, these methods often face significant challenges related to computational complexity, especially as the circuit's size and complexity grow [23]. In contrast, side-channel analysis (SCA) offers a promising alternative by evaluating differences in side-channel signatures, such as path delay and dynamic current between the target circuit and a reference design. Therefore, authors in [8] used critical path analysis to generate test vectors, maximizing the side-channel sensitivity and significantly improving both side-channel sensitivity and test generation time. While SCA has demonstrated potential in detecting HTs, its effectiveness is heavily impacted by noise and variations from environmental factors, such as temperature fluctuations or fabrication inconsistencies, which can obscure Trojan-induced anomalies [26]. The shortcomings of traditional techniques have motivated researchers to investigate the use of ML and DL–based methods in HT detection tasks.

B. ML/DL-BASED HTs DETECTION

The emphasis on traditional ML/DL-based methods for HT detection has received massive attention from the community [1], [2], [5], [6], [7], [31], as shown in Table 1. ML/DL, as a data-driven method, focuses on building computational models that learn from the features of RTL or gate-level netlist files as training samples to produce acceptable predictions. For instance, a transfer learning-based method [6] is utilized to detect Trojans in previously unexposed benchmark chips. On the other hand, authors in [1] used ZSL-based pre-trained GCN and metric learning-based methods to detect HT. Moreover, recent research has also focused on developing golden-free HT detection techniques without relying on a reference model [1], [6], [7]. However, despite the great success of these methods in HT detection, they have a few limitations, such as requiring massive amounts of high-quality label data for effective training,

extended training times to achieve accurate prediction, limited generalization to novel or unseen HTs, and insufficient explainability of the detected HT. For instance, an ML/DL model trained on specific HTs may struggle to generalize its detection to unseen HTs. However, another key issue is that many existing ML/DL-based methods rely on ad-hoc feature selection, often based on expert knowledge [8], [35], which can limit the model's generalization across various HTs and designs. Additionally, most ML/DL models operate as *"black boxes,"* providing detection outcomes without offering insight into the reasoning behind the decision or the localization of the Trojan within the design. Furthermore, although advanced ML/DL algorithms [1], [29], [31] have been proposed to address some of these limitations, they are often hindered by poor time efficiency, which limits their feasibility for large-scale or real-time applications.

C. LARGE LANGUAGE MODELS IN HTs DETECTION

Limited research has been conducted to address these research gaps. Recently, pre-trained LLM models (e.g., GPT-3.5 Turbo, Meta Llama 3.1, etc.) have been developed to excel at a wide range of complex predictive tasks with/without fine-tuning these models on datasets associated with various tasks [9], [10], [36]. LLMs possess advanced contextual understanding and can leverage vast pre-trained knowledge, which can be adapted to specific tasks with minimal additional training data. Furthermore, it can generate rationales to provide sophisticated reasoning or explanations for a given task (e.g., HT detection).

Recent studies have started exploring the potential of LLMs for hardware security tasks [11], [12], [13], [33], [34], [37], [38], [39], as shown in Table 1. For instance, authors in [37] proposed the Vul-FSM database and SecRT-LLM framework, demonstrating the effectiveness of LLMs in generating and detecting hardware vulnerabilities. On the other hand, Saha et.al [40] investigated the integration of LLMs into SOC security verification, highlighting their adaptability and scalability in this domain. Indeed, authors in [38] and [39] explored LLMs for offensive hardware security, explicitly focusing on HT creation and insertion. Moreover, authors [11] applied pre-trained LLMs (e.g., GPT-3.5 Turbo and GPT-4o-mini) as anomaly detection rules, enabling accurate identification and correction of Trojan-impacted nodes in A/MS circuits. In contrast, Wan et al. [34] employed enterprise API-based LLMs for HT detection in RTL-level benchmarks sourced from Trust-Hub. However, their evaluation relied primarily on synthetic datasets, limiting their findings' real-world applicability and generalizability. Moreover, their approach did not address the diversity of Trojan types and lacked detailed, human-readable explanations for detections, which are critical for establishing transparency and trust in the IC supply chain. On the other hand, Hayashi et al. [12] applied a combination of open-source LLMs- and ML-based methods for HT detection tasks. However, their approach did not leverage LLMs for explainability, such as localizing Trojans within flagged designs. In addition, their work did not address scalability or outline future directions such as evaluating the proposed open-source LLM-based approach in Trojan detection tasks. Similarly, Faruque et al. [13] used pre-trained LLMs (e.g., GPT-4o, Gemini 1.5 Pro, and LLaMA 3.1) in a zero-shot setting for HT detection. However, their work has several limitations. Notably, their method does not utilize the rich semantic knowledge embedded in LLMs to generate human-centric explanations, that is, to articulate *why a specific trigger or payload was classified as malicious?* This lack of interpretability hinders the trust, auditability, and usability of the detection system for hardware designers, verification engineers, and security auditors. Furthermore, the use of LLMs in a purely zero-shot fashion, without domain-specific fine-tuning or in-context learning (ICL), raises concerns about robustness when encountering novel or obfuscated Trojan variants. Leveraging ICL strategies such as one-shot or few-shot prompting could improve adaptability and reasoning under limited supervision. In contrast, our proposed method overcomes these limitations by incorporating a wide range of ICL scenarios, such as zero-shot, few-shot, and one-shot learning, validation with comprehensive datasets, addressing a broader range of Trojan types, and providing human-centric explanations that offer detailed rationales for each detection. Furthermore, most of these works are restricted to using only enterprise API-based LLMs. In contrast, we evaluate our proposed method using enterprise API-based and open-source diverse LLM models, ensuring a more comprehensive and adaptable validation framework.

III. THREAT MODEL

The threat model we consider for HT in this paper is inspired by [1] and [8]. An adversary can insert HT at various stages in the IC supply chain, such as design specification, implementation, validation, synthesis, fabrication, testing, etc. This work mainly focuses on detecting HTs during the pre-silicon phases, such as during the HDL programming phase of the IC design. Specifically, we aim to detect HT during the HDL programming phase of the IC design without requiring a golden reference model. A hardware Trojan is a malicious modification of IC designs, which typically consists of two main components: the trigger and the payload. The trigger is a specific condition or set of conditions (e.g., rare signals, rare transitions, etc.) that activate the Trojan, while the payload is the malicious action carried out once the trigger conditions are met. These Trojans are stealthy; thus, traditional functional validation may not detect HTs [1], as low-overhead safety tests are preferred [1]. As a result, when the Trojan's trigger is activated, the payload allows malicious activity, which may lead to performance degradation, information leakage, or erroneous execution. This work considers the HT, which modifies the functionality under various trigger conditions.

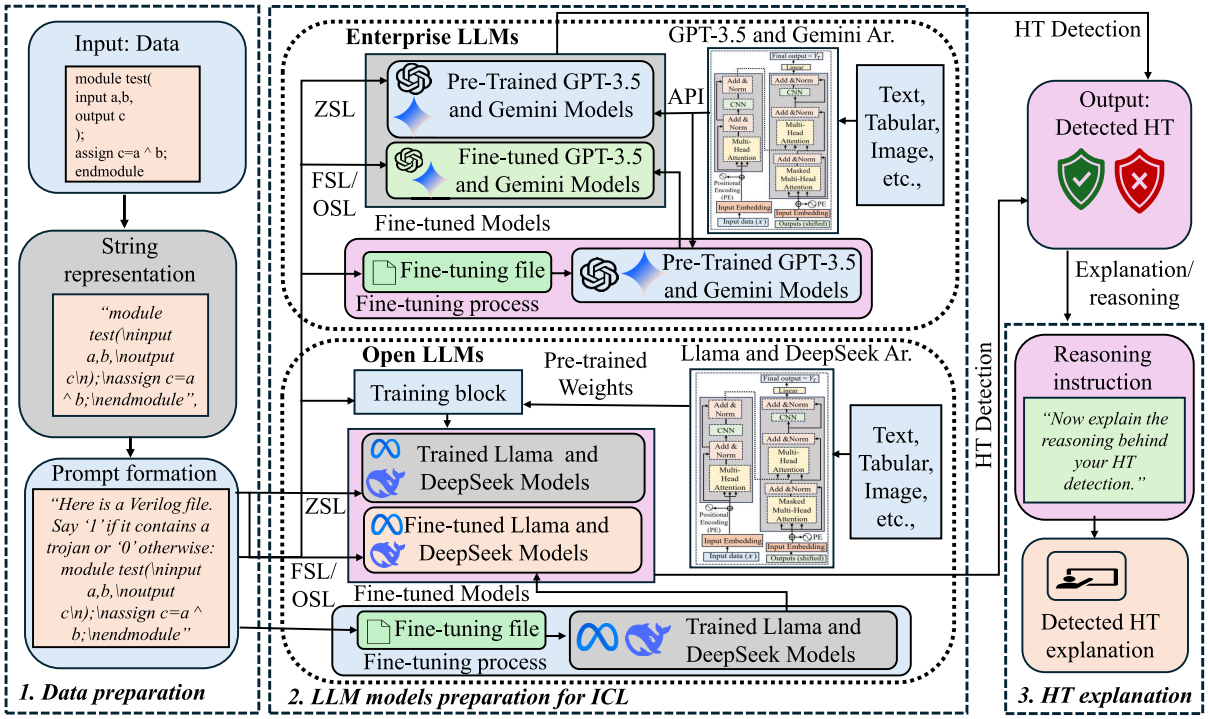


FIGURE 1. Our proposed HT detection and explanation framework, where architecture denoted as (*Ar*).

IV. PROPOSED METHODOLOGY

In this section, we present the details of our proposed methodology for the HT detection task by leveraging the knowledge of large pre-trained foundation LLM models without training the model from scratch. More specifically, we explain the architecture of the open-source pre-trained DeepSeek-V2 and Meta Llama 3.1 models and enterprise-API-enabled GPT-3.5 Turbo and Gemini-1.5 Pro models, then present how to fine-tune and use them with ICL settings, and several steps such as data preparation pipeline, prompt formulation, and explanation generation of the detected HT to build the proposed HT detection and explanation method, as shown in Figure 1. The details are as follows.

A. LARGE LANGUAGE MODELS FOR HT DETECTION

1) OPEN-SOURCE LLM MODELS IN HT DETECTION TASKS

DeepSeek V2 [16] and Llama 3.1 [15] are open source multi-modal LLMs. DeepSeek-V2 uses a Transformer [41] decoder with self-attention mechanisms to dynamically weigh the importance of different parts of the input data, but without an encoder attention part. This architecture allows the model to capture intricate patterns and dependencies, facilitating nuanced analysis critical for various applications (e.g., HT detection). DeepSeek-V2 is trained with massive amounts of text, code, image, etc., datasets and has 7 billion parameters and 32 layers containing 32 attention heads. On the other hand, Llama 3.1 is an auto-regressive language model built on an optimized Transformer architecture [41] designed for efficiency and scalability. It generates outputs sequentially by predicting the next token in a sequence. The model incorporates supervised fine-tuning to enhance its understanding of specific tasks by training on curated datasets and reinforcement learning with human feedback to align its behavior with human preferences, ensuring helpfulness and safety. These features make Llama 3.1 versatile and effective for various applications (e.g., HT detection tasks). Llama 3.1 is trained with massive amounts of text, code, etc., datasets and has 8 billion parameters with a context length of 128k. Llama 3.1, with its open-source pre-trained weights, is available for commercial and research purposes. To adopt a pre-trained Llama 3.1 LLM model for any downstream task (e.g., HT detection), we fine-tune the open-source weights on specialized datasets (e.g., HT detection), as shown in Figure 1.

2) ENTERPRISE-API-ENABLED LLM MODELS IN HT DETECTION TASKS

GPT-3.5 Turbo [10] is a compact OpenAI model designed to offer high performance while maintaining efficiency in resource usage. It utilizes a decoder-only Transformer-based architecture [41] with self-attention to dynamically weigh the importance of different parts of the input data, but without an encoder attention part. This architecture allows the model to capture intricate patterns and dependencies, facilitating nuanced analysis critical for various applications (e.g., HT detection). The model is optimized for multi-task learning and fine-tuned on extensive datasets spanning text, code, and other structured data. Its use of few-shot learning capabilities makes it ideal for a variety of tasks, such as natural language understanding and generation. On the other hand,

Gemini-1.5 Pro [14] is part of Google DeepMind's next-generation language models, offering multi-modal capabilities with advanced reasoning and multi-task abilities. Gemini 1.5 Pro employs a hybrid Transformer [41] architecture that integrates both encoder and decoder layers, allowing for highly flexible input-output transformations across a wide range of modalities (e.g., text, images, and structured data). Gemini-1.5 Pro is trained on a diverse range of multi-modal datasets. GPT-3.5 Turbo and Gemini-1.5 Pro are proprietary models and only accessible via APIs provided by their companies.

The difference between enterprise API-based pre-trained LLMs (e.g., GPT-3.5 Turbo and Gemini-1.5 Pro) and open-source LLMs (e.g., DeepSeek-V2 and Llama 3.1) for hardware Trojan detection lies in customization and accessibility. GPT-3.5 Turbo and Gemini-1.5 Pro offers ease of use, scalability, and high generalization, but it has limited customization options, as the underlying model architecture and training pipeline remain proprietary [42]. In contrast, DeepSeek-V2 and Llama 3.1 provides complete access to model weights, enabling tailored fine-tuning for specific tasks (e.g., HT detection), but requires significant computational resources and extended fine-tuning time [43]. Moreover, enterprise API-based LLM models offer better optimization for inference efficiency and integration, while open-source models provide more control at the cost of requiring more development effort [42], [43]. Indeed, enterprise API-based LLM models can be updated, replaced, or discontinued by the provider without user control, potentially disrupting workflows and compatibility [44]. In contrast, open-source LLMs offer full transparency and control, as users can download, modify, and deploy the model independently. This ensures long-term stability and adaptability for specific tasks (e.g., HT detection) without being affected by changes from a third-party provider [43], [44].

B. DATA PREPARATION AND PROMPT FORMULATION FOR LLM MODELS FOR HT DETECTION

This Section presents the data-preprocessing steps and prompt formalization for LLMs in HT detection tasks. The details are described below:

1) DATA PREPARATION

To use an LLM for HT detection tasks, the data (e.g., RTL files (Verilog) of the circuit) must be transformed into a string representation. Typically, when prompting an LLM, there is a template used to convert the inputs into one natural-language string and to provide the prompt itself (e.g., the string for the Verilog code *"module test(\n input a,b;\n output c;\n);\nassign c = a ^ b; \n endmodule"*).

2) PROMPT FORMULATION FOR ENTERPRISE AND OPEN-SOURCE LLM MODELS FOR HT DETECTION

The performance of LLMs is susceptible to the precise details of the natural-language input [45]. Prompt engineering is an essential technique for guiding LLMs to perform specific tasks. By carefully crafting prompts, we can extract relevant information and classifications regarding the presence of HTs from HT data (e.g., Verilog code). Effective prompts are designed to give the LLM the necessary context and instructions to perform HT detection and provide meaningful explanations of the given circuit. Figure 2 shows a simple prompt format for HT detection tasks that require minimal human effort to apply to new HT classification tasks. To effectively guide enterprise API, also known as proprietary LLMs (e.g., GPT-3.5 Turbo and Gemini-1.5 Pro) and open-source LLMs (e.g., Llama 3.1 and DeepSeek-V2) in HT detection tasks, we formalize the prompt structure by incorporating different prompt elements, such as task (objective of the prompt), context (providing background information to the model to understand the task), and format (specifying the desired output format, such as HT detection (e.g., Yes: '1' or No:'0') or explanations of the detected HT), ensuring structured reasoning through the chain-of-thought (CoT) prompting technique [36], [45], [46]. For instance, the task explicitly defines the objective, instructing the model to analyze HT data (e.g., Verilog code) and determine whether a hardware Trojan is present. The context provides essential background information, including HT characteristics and common insertion patterns distinguishing Trojans from benign data. This background knowledge helps the model establish a reasoning path when evaluating circuits. To enforce structured reasoning, the format specifies the expected output, such as binary classification (Yes: '1' or No: '0') for HT detection, followed by an explanation justifying the decision. The CoT prompting technique guides the model in decomposing the detection process into step-by-step logical reasoning, including analyzing control and data flow, identifying suspicious modifications, and correlating anomalies with known Trojan patterns. This structured approach improves classification accuracy and enhances interpretability by providing human-readable justifications for the detection results. By formalizing the prompt with these elements, we ensure minimal human intervention while enabling adaptable and scalable HT detection across different circuit designs. It is important to note that we chose the CoT prompting method because it is well-established for complex reasoning tasks and aligns with our objective of enhancing detection accuracy and interpretability [47]. CoT enables the model to systematically analyze HT data (e.g., VHDL code) by breaking the detection process into logical steps, facilitating a more structured and explainable decision-making process.

C. ENTERPRISE AND OPEN-SOURCE LLM MODELS FOR HT DETECTION TASKS USING ICL

To utilize the enterprise API-based GPT-3.5 Turbo and Gemini-1.5 Pro and open-source-LLM-based Llama 3.1 and DeepSeek-V2 LLM models for hardware trojan detection tasks, we consider several ICL strategies: zero-shot learning

(ZSL), one-shot learning (OSL), and few-shot learning (FSL) strategy. These learning methods send context and prompt formatted samples to OpenAI's API for the GPT-3.5 Turbo, and Google's API for Gemini-1.5 Pro models and fine-tuned open-source Llama 3.1 and DeepSeek V2 trained weights for the both model. The context passed to either the enterprise API or fine-tuned open-source pre-trained weights remains constant throughout all the learning methods, but our prompts and models change depending on the learning method. For ZSL with enterprise API (e.g., OpenAI and Google), we use the pre-trained LLM models and feed the historical HT benchmark data directly, followed by our prompt-formatted sample to detect HT without adjusting these LLM models' parameters. On the other hand, for ZSL using the open-source Llama 3.1 and DeepSeek-V2 LLM model, we fine-tuned the pre-trained LLM model weights on domain-specific datasets of circuit benchmarks. The fine-tuning process trains the model pre-trained weights to learn the general knowledge of benign and Trojan-inserted designs by encoding these benchmarks' key properties and patterns. This pre-trained knowledge allows the model to analyze previously unexposed benchmarks without additional fine-tuning during inference. We use 20 epochs, set the temperature to 0.3, and limit the output tokens to 128k for fine-tuning the pre-trained open-source models' weights and parameters in the HT detection task. On the other hand, for OSL and FSL, we provide prompt formatted samples on the enterprise API-based and open-source-weights-based LLM models, using domain-specific knowledge (e.g., HT benchmark data). For instance, in OSL, we provide only one example (e.g., a Trojan-infected sample/benign sample), followed by our prompt formatted sample to the enterprise API and open-source LLM models. On the other hand, for the FSL, we provide a few examples from these benchmarks, followed by our prompt formatted samples to fine-tune all LLM models to classify the sample as Trojan-infected or benign. The details of OSL and FSL-based fine-tuning strategies are described below.

D. FINE-TUNING ENTERPRISE AND OPEN-SOURCE LLM MODELS FOR HT DETECTION TASK USING FSL AND OSL

This phase aims to adapt and fine-tune the enterprise API-based LLM models (i.e., GPT-3.5 Turbo and Gemini-1.5 Pro) and open-source weights-based LLM models (i.e., Llama 3.1 and DeepSeek-V2) using domain-specific knowledge via FSL and OSL. FSL and OSL are extreme transfer learning variants, which rely on training the model with one specific type of example (i.e., OSL) or a few samples (i.e., FSL) for a particular task [48]. Initially, the enterprise API-enabled and open-source LLM models are trained on massive amounts of data (e.g., texts, codes, etc.), capturing wide-ranging features and patterns. This model can then be specialized for specific contexts, such as HT detection tasks. Fine-tuning is a powerful process that effectively utilizes a pre-trained model for specific contexts (e.g., HT detection tasks). By fine-tuning, we adjust these LLM models (e.g., enterprise and

```
Does the provided Verilog code indicate any signs
of an HT, such as unauthorized data manipulation,
hidden triggers, backdoors, or anomalies? Respond
with '1' if you detect Trojan, and '0' if the code
is clean.
```

FIGURE 2. Prompt template for HT detection of a Verilog code in ICL context.

open-source) parameters for the HT detection task, allowing the model to tailor its vast pre-existing knowledge toward the requirements of the HT detection task. As shown in Figure 1, to fine-tune the enterprise API-based (i.e., GPT-3.5 Turbo and Gemini-1.5 Pro) and open-source weights-based (i.e., Llama 3.1 and DeepSeek-V2) LLM models for the HT detection task, we provide the designed prompt, which contains Trojan-infected and benign class samples in the FSL setting and one specific type of example class (e.g., a Trojan-infected / benign sample) in the OSL setting.

Furthermore, to ensure consistency and fair comparison across these two settings, we use the same unified prompt as in the ZSL setting for both OSL and FSL, as illustrated in Figure 2. Standardizing the prompt isolates the effect of in-context examples by removing confounding factors from prompt variation. This design choice directly compares how different ICL paradigms (e.g., ZSL, FSL, and OSL) influence the model's performance on the same task formulation, thereby strengthening the validity of our experimental results. In few-shot cases, we prepend the prompt with multiple labeled examples, each comprising Verilog snippets and their corresponding HT labels, to guide the model's decision. For instance, in an FSL setting, we include a few labeled HT examples before querying the model on an unlabeled input. This consistent format supports ICL inference uniformly, enabling scalable and low-effort adaptation across various HT detection tasks. We adopt these enterprise and open-source (e.g., GPT-3.5 Turbo, Gemini-1.5 Pro, Llama 3.1, and DeepSeek-V2) LLM model's default hyperparameters without additional parameter tuning for fine-tuning. We fine-tune enterprise API-based GPT-3.5 Turbo through the OpenAI API and Gemini-1.5 Pro through the Google-provided API, a capability introduced in August 2023 [49] by OpenAI and September 2024 [14] by Google. More specifically, we perform actual model fine-tuning (updating model weights) using OpenAI and Google's fine-tuning endpoint, not just prompt engineering. Our methodology follows OpenAI and Google-recommended fine-tuning practices, which include preparing training data in *JSONL* format and using the fine-tuning API endpoint with our custom training dataset. The process involved true parameter updates, distinct from ICL or prompt engineering approaches, similar to approaches demonstrated in [50], [51], [52]. For this, we set $K = 10, 12, 14, 16$ shots in the FSL setting and use 10 epochs for fine-tuning both learning settings. We set the temperature to 0.3 and limited the output tokens to 900 for the GPT-3.5 Turbo and Gemini-1.5 Pro LLM models. On the other hand, for open-source weights-based (i.e., Llama 3.1 and DeepSeek-V2) LLM models, we perform traditional

```
Now, provide a brief reason for this
classification. Include <line numbers> where
the trojan occurs, <wire names> that are
involved with the trojan and the type of
<trojan trigger>.
```

FIGURE 3. Template for explaining the reasoning of detected HT.

fine-tuning of the Llama 3.1 with 8B-parameter model and DeepSeek-V2 with 7B-parameter model, adjusting its weights with a learning rate of 2×10^{-5}, 20 epochs for ZSL, and 10 epochs for OSL and FSL (K=10, 12, 14, 16), using the AdamW optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) [53], by following the established fine-tuning protocols as recommended in [15] and [54]. We set the temperature to 0.3 and limited the output tokens to 128k for the Llama 3.1 and DeepSeek-V2 LLM models. Once the fine-tuning process is complete, we use the fine-tuned model to evaluate FSL and OSL performance, which involves feeding a prompt formatted sample to detect HT.

E. EXPLANATION/REASONING OF THE DETECTED HT USING ENTERPRISE AND OPEN-SOURCE LLM MODELS

To generate interpretable responses from the detected HT, we provide the LLM with meaningful contextual prompts and instructions to give reasoning/explanation for its classification of any given circuit. For this, prompt engineering techniques are employed to refine queries of the detected HT, which guide the LLM toward a precise explanation. Specifically, we use the CoT prompting technique similar to the HT detection tasks, which ensures that the LLM models systematically break down their reasoning logically and interpretably. CoT prompting enables the LLM models to follow a sequential reasoning approach by first determining whether the circuit contains a Trojan or no Trojan, then mapping the detected Trojan to specific elements such as line numbers, wire names, and Trojan triggers, and finally, synthesizing a structured explanation. Therefore, when an HT is detected, the model is prompted to analyze the features from the HT data (e.g., line numbers, wire names, Trojan trigger type, etc.). It extracts line numbers where malicious logic appears, wire names contributing to the Trojan's activation, and the Trojan trigger type (e.g., combinational, sequential, etc.). to classify the HT operational mechanism. This stepwise breakdown ensures that explanations are aligned with the detection decision and transparent and interpretable, thus providing the detected HT's trustworthiness. The explanation/reasoning prompt template is shown in Figure 3. Note, for the reasoning/explanation of the detected HT using enterprise LLMs, we set the temperature to 0.2 and limited the output tokens to 100 for the GPT-3.5 Turbo and Gemini-1.5 Pro LLM models. On the other hand, for the open-source LLM models (i.e., Llama 3.1 and DeepSeek-V2), we set the temperature to 0.3 and limit the output tokens to 1000.

TABLE 2. Sizes of benchmarks from ISCAS [18] and Trust-Hub [17].

Benchmarks	Inputs	Outputs	No. of Signals	LOC
AES-T900	128	128	286	92
AES-T1000	128	128	293	93
AES-T1100	128	128	284	92
AES-T1200	128	128	289	96
AES-T1300	128	128	290	102
AES-T1400	128	128	290	102
AES-T1600	128	128	292	92
AES-T1700	128	128	295	104
c2670	233	140	1193	1482
c3540	50	22	1669	1877
c5315	178	123	2406	2615
c6288	32	32	2406	2685
c7552	207	108	3512	3945
s13207	31	121	7951	3684
s15850	14	87	9772	4011
s35932	35	320	16065	39744

V. RESULTS AND DISCUSSION

This section discusses the results obtained from our proposed method performance in HT detection tasks, along with the explainability of the detected trojans.

A. DATASETS AND BASELINE

This section provides an overview of the datasets and baseline setup that we used to validate our proposed approach in HT detection tasks.

1) DATASETS

We evaluate our proposed framework using on diverse benchmarks from Trust-Hub [17], ISCAS (both ISCAS'85 and ISCAS'89) [18] datasets which comprised of both combinational and as well as sequential circuits. Table 2 shows the statistics about the benchmark circuits, such as the number of input and output variables for the top-level module, the number of signals, and the lines of code (LOC) for each Benchmarks. The benchmarks from Trust-Hub are AES-T900, AES-T1000, AES-T1100, AES-T1200, AES-T1300, AES-T1400, AES-T1600, and AES-T1700, and ISCAS is c2670, c3540, c5315, c6288, c7552, s13207, s15850, and s35932, respectively. We chose these benchmarks since they are widely used by SOTA HT detection methods [1], [2], [5], [6], [7], [8], [32], [55].

We investigate various kinds of Trojan types with combinational, sequential, and hybrid triggers using different attributes (e.g., rare signals, nonrare signals, rare branches, synchronous and asynchronous counter, etc.) similar to the work [1]. Rare signals are infrequently changed state under normal circuit operations. For instance, a rarely used error signal might be a trigger, ensuring the Trojan remains dormant during functional testing. Trojans relying on rare signals are particularly stealthy because their activation conditions are seldom encountered, reducing the likelihood of detection during standard validation [27], [56]. On the other hand, nonrare signals toggle more frequently (e.g., a clock-enabled signal or a data bus line). Trojans triggered

by non-rare signals rely on embedding malicious logic into normal behavior, masking their presence by blending with regular activity. This kind of trigger is less stealthy than a rare signal trigger [27], [56]. In contrast, rare branches occur in conditional logic paths executed during regular operations. Trojans using rare branches ensure that their payload executes only under unique conditions, making them effective for avoiding detection during functional validation [27]. However, a Trojan trigger using a synchronous counter relies on a counter that increments on every clock cycle [1]. The trigger activates when the counter reaches a specific value. Meanwhile, asynchronous counters rely on specific events rather than clock cycles to increase. For instance, a counter might track the number of times a rare signal toggles. The Trojan activates when the counter reaches a particular threshold, requiring precise event occurrences [57].

Figure 4 shows an example Trojan from our experiment that employs a hybrid trigger mechanism consisting of combinational (e.g., rare (r1) and nonrare (r2) signals) and sequential (e.g., synchronous (Counter 1) and asynchronous counters (Counter 2)) events. A specific value of Counter 2 activates the trigger for this given Trojan example. It is important to note that Trust-Hub benchmarks originally use explicit HT names (e.g., 'Tj_Trig,' 'Trojan_Trigger,' etc.,), as these are standard identifiers in this dataset. However, we renamed signals to generic terms (e.g., 'Tj_Trig' to 'clk_en'), aligning with ISCAS's subtle triggers to test deeper detection capabilities. Renaming explicit signals ensured detection relied on semantic analysis, not trivial string matching. Once the LLM detects the HT (with a generic name) and provides a human-centric reasoning /explanation, we revert it to its original name. For consistency with the past studies, we use the original signal names in the subsequent sections for these Trust-Hub benchmarks with explicit names by replacing the generic ones in the final output.

TABLE 3. Comparison of different datasets for SOTA HT detection methods with our proposed method.

HT detection methods	Benchmarks	
	Trust-Hub	ISCAS
TD-Zero [1]	√	√
SVM [5]	√	X
HTnet [6]	√	X
COTD [7]	√	X
Our proposed	√	√

2) BASELINE SETUPS

Using the following configurations, we evaluate and compare our method performance with SOTA of the following four ML/DL-based HT detection schemes such as TD-Zero [1], HTNet [6], COTD [7], and SVM [5]. For TD-Zero [1], we use an ML model pre-trained with supervised learning of known scenarios (i.e., known HTs) and perform HT detection using unsupervised learning in unknown scenarios (i.e., new benchmarks with unknown HTs) [1] using the metric learning task is used to measure the similarity between test and malicious samples to make detection. To train the TD-Zero for HT detection tasks, we use the same model architectures mentioned in [1]. For instance, the neural network consists of three consecutive graph convolutional layers (GCLs), each followed by a dropout and max-pooling layer to avoid overfitting problems. We use 50 epochs with learning rate 10^{-3} to train this model. However, for SVM [5], we use a supervised learning-based HT detection method using a support vector machine (SVM). For this, we use a radial basis function kernel. While for HTNet [6], we use unsupervised learning-based HT detection. HTNet leverages a neural network with a k-means clustering technique. HTnet consists of two Convolutional Layers with ReLU activations, each immediately followed by dropout and max-pooling mechanisms to avoid overfitting. We use a Stochastic Gradient Descent (SGD) optimizer with a learning rate 0.001 and use 30 epochs to train the HTnet for HT detection tasks. On the other hand, for COTD [7], we use an unsupervised learning-enabled k-means clustering technique, which uses Euclidean distance to classify the Trojans.

Furthermore, we train these models by using diverse benchmark datasets from ISCAS and Trust-Hub. For TD-Zero, we use Trust-Hub (e.g., AES-T1100, AES-T1200, AES-T1600, and AES-T1700) and ISCAS (e.g., ISCAS is c2670, c3540, c5315, c6288, c7552, s13207, s15850, and s35932) benchmarks with 200 Trojan-infected and benign samples, focusing on known and unseen HTs. For SVM, we train the model on Trust-Hub benchmarks with Trojan features (e.g., rare signals), approximately 500 samples. In contrast, for HTNet, we utilize 400 Trust-Hub RTL samples, while for COTD, we use 300 Trust-Hub samples, emphasizing controllability and observability. Furthermore, our proposed method evaluates more diverse benchmark datasets than other SOTA works in HT detection tasks, as shown in Table 3. Indeed, it is worth mentioning that we evaluate our proposed method with more benchmarks from the Trust-Hub dataset than the TD-Zero work [1]. For instance, TD-Zero validated its approach with four Trust-Hub benchmarks: AES-T1100, AES-T1200, AES-T1600, and AES-T1700, with 200 Trojan-infected and benign samples. In contrast, our proposed method is validated with eight Trust-Hub benchmarks (e.g., AES-T900, AES-T1000, AES-T1100, AES-T1200, AES-T1300, AES-T1400, AES-T1600, and AES-T1700) with a broader set of 400 RTL samples from Trust-Hub benchmarks.

B. EXPERIMENTAL SETUP AND EVALUATION

To use the enterprise API-based pre-trained LLM models (e.g., GPT-3.5 Turbo and Google-1.5 Pro) for ICL settings, we use OpenAI provided API of the GPT-3.5 Turbo and Google provided Gemini-1.5 Pro LLM model. *Note, we used the GPT-3.5 turbo model instead of the GPT-4o model since the GPT-3.5 turbo model is more cost-effective and faster than the GPT-4o model [10], [58], which makes it a preferable choice for rapid experimentation and iteration in*

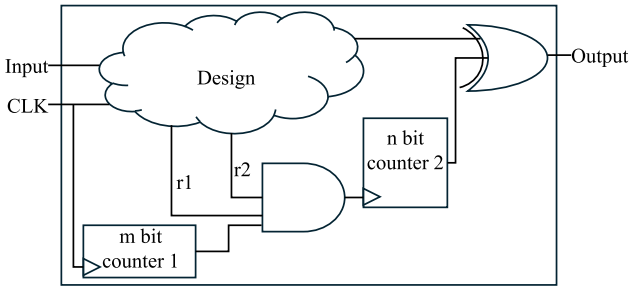


FIGURE 4. A sample Trojan from our experiment that applies a hybrid trigger mechanism.

HT detection tasks. On the other hand, to use the open-source weights Llama 3.1 LLM and DeepSeek V2 models for ICL settings, we use the Hugging Face [59] and vLLM [60] library. *Note, we used the Llama 3.1 model with 8 billion parameters and DeepSeek-V2 7 billion instead of their larger counterparts with 40 plus billion parameters since models with more than 8 billion for Llama 3.1 and 7 billion for DeepSeek-V2 models parameters often exceed the memory and computational capacity of standard GPUs [61], making them impractical for on-premises deployment. Furthermore, Llama 3.1 and DeepSeek-V2, with 8 and 7 billion parameters, respectively, balances performance and resource requirements, allowing for real-time inference and fine-tuning directly on locally available GPUs, ensuring privacy, cost-effectiveness, and independence from cloud services [61], [62].* For Llama 3.1 and DeepSeek-V2 models, we performed traditional fine-tuning by adjusting its weights with a learning rate of 2×10^{-5}, 20 epochs for ZSL, and 10 epochs for OSL and FSL (K=10, 12, 14, 16), using the AdamW optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) [53], by following the established fine-tuning protocols as recommended in [15] and [54]. The dataset mirrored that of GPT-3.5 Turbo and Gemini-1.5 Pro (400 samples). On average, in the single-shot ICL setting, the total prompt consumes approximately 1,200–1,500 tokens, depending on the size of the design for each benchmark. All experiments are conducted using an Intel Core i9 Processor and 128GB RAM option with NVIDIA GeForce RTX 3090 Ti GPU. It is important to note that we use the same experimental setup to train and validate our baseline methods.

C. PERFORMANCE METRICS

We use the most widely used performance metrics to evaluate the performance of the HTs detection method with accuracy, precision, recall, F1-score, true positive rate (TPR), true negative rate (TNR), false positive rate (FPR), and false negative rate (FNR) [1], [2], [13]. It is important to note that instead of using solely accuracy metrics for evaluation, we choose these metrics because, when creating benign and malicious sample datasets, traditional methods randomly generate rare trigger conditions for each benchmark and integrate them into the original design to form a Design Under Test (DUT) [1], [2]. However, this approach often produces imbalanced datasets, introducing data bias and resulting in a naive ML/DL model achieving accuracy near 100% by simply predicting most inputs as malicious. This demonstrates that the accuracy metric alone is insufficient for evaluating the effectiveness of the proposed method. To address this, we use these alternative evaluation metrics along with accuracy metrics in HTs detection tasks. Furthermore, these evaluation metrics assess how well an ML/DL model distinguishes between Trojan-infected and Trojan-free designs, accounting for positive and negative outcomes. For instance, precision focuses on the correctness of positive predictions, recall measures the ability to identify actual Trojans, and the F1-score balances both. TPR and TNR assess detection sensitivity and specificity, respectively, while FPR and FNR highlight potential errors in misclassification, offering insights into the model's reliability under real-world conditions where false alarms and missed Trojans can have significant consequences.

D. HT DETECTION PERFORMANCE ANALYSIS USING ICL METHOD FOR ENTERPRISE AND OPEN-SOURCE LLM MODELS

Tables 4, 5, 6, and 7 show the mean accuracy (with standard deviation values) and other performance metrics such as precision, recall, F1-score, TPR, TNR, FPR, and FNR for the HT detection task of the enterprise API-based GPT-3.5 Turbo and Gemini-1.5 Pro LLM models and open-source Llama 3.1 and DeepSeek-V2 LLM models. These results are reported across various ICL settings: few-shot, one-shot, and zero-shot evaluated on the Trust-Hub and ISCAS '85/'89 benchmarks, along with both benchmarks together. It is important to note that the standard deviations for accuracy metric are computed over multiple runs ($n = 5$, where n is the number of experiment run), each using different random seeds, for both enterprise and open-source LLMs under zero-shot, one-shot, and few-shot settings. Additionally, it is also important to note that in our work, the ZSL-based method always involves detecting unseen HTs, as we directly feed the historical HT benchmark data to the model. By "unseen" we refer to benchmarks with different sources and functionalities. From Tables 4, 5, 6, and 7, we observe that the FSL-based fine-tuned method achieves higher detection performance than the OSL-based fine-tuned and ZSL-based without a fine-tuned model. For instance, the OSL-based enterprise GPT-3.5 Turbo LLM model achieves a mean HT detection accuracy of 1.0, 0.94, and 0.97 for the Trust-Hub, ISCAS' 85/'89, and both (Trust-Hub and ISCAS' 85/'89) benchmarks, which is 1.03×, and 1.05× higher than OSL and ZSL-based approaches for Trust-Hub, 1.17×, and 1.11× higher than OSL and ZSL-based approaches for ISCAS' 85/'89, and 1.06×, and 1.21× higher than OSL and ZSL-based methods for both benchmarks, respectively. A similar phenomenon is observed for the other LLM models (e.g., Gemini 1.5 Pro, Llama 3.1, and DeepSeek-V2) for all these benchmarks.

TABLE 4. Performance of HT detection tasks for different benchmarks using the GPT-3.5 Turbo LLM model. The notation X/Y/Z represents the Trust-Hub/ISCAS/both and accuracy with standard deviation values.

Method	Accuracy	Precision	Recall	F1-Score	TPR	TNR	FPR	FNR
Few Shot	1.0±0.02/0.94±0.01/0.97±0.02	1.0/0.98/1.0	1.0/0.91/0.96	1.0/0.94/0.98	1.0/1.0/0.98	1.0/1.0/1.0	0.0/0.0/0.0	0.0/0.02/0.0
One Shot	0.98±0.01/0.91±0.03/0.93±0.02	0.97/0.93/0.98	0.95/0.86/0.9	0.96/0.87/0.93	0.98/0.92/0.91	1.0/0.98/0.99	0.0/0.04/0.01	0.0/0.1/0.05
Zero Shot	0.92±0.02/0.85±0.01/0.9±0.01	0.93/0.83/0.88	0.90/0.81/0.88	0.9/0.8/0.88	0.92/0.89/0.94	0.94/0.8/0.89	0.0/0.27/0.18	0.0/0.22/0.16

TABLE 5. Performance of HT detection tasks for different benchmarks using enterprise API-enabled Gemini-1.5 Pro LLM model. The notation X/Y/Z represents the Trust-Hub/ISCAS/both and accuracy with standard deviation values.

Method	Accuracy	Precision	Recall	F1-Score	TPR	TNR	FPR	FNR
Few Shot	0.95±0.00/0.9±0.01/0.96±0.00	0.96/0.93/0.97	0.96/0.88/0.94	0.96/0.92/0.96	0.98/0.97/0.96	0.99/0.98/0.98	0.01/0.02/0.01	0.02/0.04/0.03
One Shot	0.94±0.03/0.89±0.01/0.91±0.03	0.93/0.92/0.93	0.95/0.81/0.88	0.93/0.82/0.91	0.96/0.9/0.91	0.97/0.95/0.96	0.03/0.04/0.04	0.03/0.13/0.08
Zero Shot	0.9±0.01/0.8±0.01/0.83±0.01	0.91/0.76/0.84	0.9/0.76/0.84	0.90/0.77/0.84	0.91/0.86/0.93	0.93/0.7/0.84	0.03/0.30/0.23	0.05/0.28/0.20

TABLE 6. Performance of HT detection tasks for different benchmarks using open-source DeepSeek-V2 LLM model. The notation X/Y/Z represents the Trust-Hub/ISCAS/both and accuracy with standard deviation values.

Method	Accuracy	Precision	Recall	F1-Score	TPR	TNR	FPR	FNR
Few Shot	0.93±0.02/0.85±0.01/0.91±0.01	0.95/0.90/0.92	0.94/0.85/0.95	0.96/0.89/0.93	0.94/0.96/0.93	0.94/0.91/0.93	0.04/0.1/0.07	0.03/0.11/0.09
One Shot	0.91±0.00/0.83±0.02/0.89±0.01	0.93/0.89/0.92	0.9/0.79/0.87	0.93/0.8/0.86	0.9/0.86/0.83	0.93/0.89/0.88	0.09/0.18/0.12	0.06/0.15/0.1
Zero Shot	0.88±0.02/0.81±0.01/0.86±0.00	0.92/0.73/0.83	0.86/0.72/0.8	0.88/0.74/0.81	0.86/0.8/0.84	0.85/0.76/0.81	0.14/0.38/0.32	0.15/0.32/0.27

TABLE 7. Performance of HT detection tasks for different benchmarks using open-source Llama 3.1 LLM model. The notation X/Y/Z represents the Trust-Hub/ISCAS/both and accuracy with standard deviation values.

Method	Accuracy	Precision	Recall	F1-Score	TPR	TNR	FPR	FNR
Few Shot	0.91±0.02/0.84±0.03/0.90±0.02	0.94/0.90/0.92	0.93/0.82/0.96	0.95/0.86/0.91	0.93/0.96/0.92	0.92/0.90/0.91	0.06/0.11/0.08	0.04/0.12/0.09
One Shot	0.90±0.01/0.80±0.02/0.87±0.00	0.91/0.87/0.91	0.88/0.63/0.82	0.93/0.70/0.80	0.89/0.83/0.80	0.91/0.87/0.86	0.11/0.21/0.14	0.09/0.18/0.13
Zero Shot	0.85±0.01/0.70±0.03/0.81±0.01	0.92/0.58/0.74	0.84/0.54/0.73	0.81/0.62/0.74	0.81/0.77/0.83	0.81/0.53/0.71	0.15/0.41/0.33	0.16/0.37/0.31

However, the enterprise API-based GPT-3.5 Turbo and Gemini-1.5 Pro models perform better than the open-source DeepSeek-V2 and Llama 3.1 LLM models for all ICL settings in HT detection tasks for all benchmarks, as shown in Tables 4, 5, 6, and 7. For instance, in the FSL, the GPT-3.5 Turbo model achieves 1.07×, 1.11×, and 1.08× higher mean detection accuracy than DeepSeek-V2 LLM model and 1.10×, 1.13×, and 1.08× higher mean detection accuracy than Llama 3.1 LLM model for the Trust-Hub, ISCAS' 85/'89, and both (Trust-Hub and ISCAS' 85/'89) benchmarks. Likewise, in Trust-Hub and ISCAS '85/'89 benchmarks, the FSL method performs better than the OSL and ZSL methods, and the GPT-3.5 Turbo model performs better than open-source Llama 3.1, and DeepSeek-V2 LLM models in HT detection tasks. Indeed, the enterprise GPT-3.5 Turbo performs better than enterprise-based Gemini models in HT detection tasks. A similar phenomenon is observed for other metrics (e.g., precision, recall, and F1-score). Indeed, the overall performance of the FSL-based GPT-3.5 Turbo LLM model is slightly better than the OSL and ZSL-based other LLM models in terms of precision, recall, F1-score, TPR, TNR, FPR, and FNR for HT detection for all datasets. This is because the FSL approach leverages a small amount of labeled data to fine-tune pre-trained LLMs, enabling better adaptation to task-specific nuances [6]. Thus, the FSL method improves the model's ability to identify subtle hardware Trojan patterns compared to the generalized knowledge in ZSL models, which rely solely on generalized pre-training or OSL models, which learn from a single example.

Furthermore, to better evaluate the capability of the OSL and FSL approaches in unseen HT detection tasks, we evaluate our proposed method using unseen (unknown) benchmarks with these two learning approaches. For instance, in the OSL-based approach, we fine-tuned our LLM model using s13207 from the ISCAS89 dataset with an injected Trojan that causes functional corruption and tested it using the AEST1100 benchmark from Trust-Hub implanted with a Trojan that causes information leakage. On the other hand, in the FSL-based approach, we fine-tuned multiple benchmarks from the ISCAS89 dataset and tested them with the Trust-Hub dataset. This strategy can remove bias from our model and test the proposed transfer learning capability's speculation capacity in hardware trojan detection. Tables 8 and 9 show the mean accuracy, precision, recall, and F1-score for our OSL-based method transferability performance for unknown HT detection using GPT-3.5 Turbo, Gemini-1.5 Pro, DeepSeek-V2 and Llama 3.1 LLM models, respectively. We observe that the OSL performance for unknown HT detection, when given the ISCAS '85/'89 benchmark samples, achieves higher detection performance than Trust-Hub benchmark samples using GPT-3.5 Turbo LLM model, as shown in Table 8. For instance, when given either a trojan-infected or trojan-free sample from ISCAS '85/'89 dataset, the detection accuracy is 0.99, which is 1.12× and 1.16× higher than when it's given

a Trojan-infected and Trojan free sample for Trust-Hub benchmark, respectively. A similar phenomenon is also observed for the other LLM models, such as Gemini-1.5 Pro, DeepSeek-V2, and Llama 3.1 LLM model, as shown in Table 8 and 9.

Moreover, Tables 10 and 11 present mean accuracy, precision, recall, and F1-score of FSL-based transferability evaluation of our fine-tuned GPT-3.5 Turbo, Gemini-1.5 Pro, DeepSeek-V2, and Llama 3.1 LLM models using Trust-Hub, and ISCAS '85/'89. We fine-tuned our model with the Trust-Hub dataset and evaluated it on the ISCAS dataset for different shots, such as 10, 12, 14, and 16. Like OSL-based performance in unknown HT detection tasks, it is observed that the ISCAS dataset achieves higher detection accuracy for different numbers of shots. For instance, using the GPT-3.5 Turbo and Gemini-1.5 Pro LLM models, the ISCAS dataset achieves an F1-score of 1.0 and 0.96 for 16 shots, which is 1.11× and 1.09× higher than the Trust-Hub dataset for the same number of shots. This suggests that while the models are highly sensitive, their precision and overall accuracy are limited when transferred to the ISCAS dataset. From this, we observe that the FSL setting detects HT is easier for the Trust-Hub dataset and, therefore, is trained on worse data and tested on better data, yielding poor results. In contrast, the performance under the ISCAS dataset represents the model fine-tuned on the ISCAS dataset and evaluated on the Trust-Hub dataset. In this scenario, the models exhibit perfect performance across all metrics: accuracy, precision, recall, and F1-score, consistently achieving a score of 1.0 for 10, 12, 14, and 16 shots. This result indicates that the model trained on the ISCAS dataset generalizes exceptionally well to the Trust-Hub dataset, maintaining complete accuracy and reliability across all shots tested. However, the HT detection performance is slightly lower with the tabular dataset in the FSL setting compared to the ISCAS and Trust-Hub datasets. A similar phenomenon is also observed for the open-source DeepSeek-V2 and Llama 3.1 LLM models, as shown in Table 11. However, we observe that the enterprise API-based GPT-3.5 Turbo and Gemini-1.5 Pro LLM models often outperforms the open-source DeepSeek-V2 and Llama 3.1 LLM models in HT detection tasks in all ICL settings. This is because enterprise models benefit from proprietary training techniques, advanced architectures, and access to vast computational resources, enabling them to generalize effectively across complex tasks (e.g., HT detection) [10], [42]. In contrast, while customizable, open-source LLM models depend on the quality of domain-specific fine-tuning, which may limit their performance if training data or resources are constrained [42], [43].

E. COMPARISON OF HT DETECTION PERFORMANCE

This section compares our proposed method with state-of-the-art ML/DL-based HT detection methods. First, we compare our proposed method with TD-Zero, which uses an ML model pre-trained with supervised learning of known scenarios (i.e., known HTs) and performs HT detection using unsupervised learning in unknown scenarios (i.e., new benchmarks with unknown HTs) [1]. After that, we compare our method with SOTA other methods, such as HTNet [6], COTD [7], and SVM [5]. It is important to note that the FSL-based method performs better than other methods (e.g., OSL and ZSL) for unseen HT tasks, as shown in Table 4 and 7; thus, we compare our results with other SOTA works for only the FSL-based approach in the rest of the paper. Furthermore, it is important to clarify that TD-Zero's implementation of ZSL differs significantly from our proposed method. TD-Zero first develops a supervised knowledge base of known scenarios (e.g., simple benchmarks with known HTs) using an ML model and performs Trojan detection using ZSL in unknown scenarios (e.g., new and complex benchmarks with unknown HTs), which requires training from scratch. In contrast, our ZSL strategy leverages pre-trained enterprise and open-source LLM models (i.e., GPT-3.5 Turbo, Gemini-1.5 Pro, DeepSeek-V2 and Llama 3.1) knowledge by directly feeding historical HT benchmark data without additional training, making it more efficient and resource-effective. Thus, an experimental comparison of these two methods will be unfair and will not offer any critical insights with scientific significance.

Furthermore, the enterprise-API-based GPT-4o-mini LLM model performs better than the enterprise-API-based Gemini LLM model and open-source DeepSeek-V2 and Llama 3.1 model for HT detection tasks in various ICL settings, as shown in Tables 4, 5, 6 7, 8, 9, 11 and 10; thus, we compare only the GPT-3.5 Turbo LLM model performance for FSL setting with other SOTA works. Figure 5(a) shows the comparison of the performance of our FSL performance with the TD-Zero using accuracy, precision, recall, F1-score, TPR, TNR, FPR, and FNR for unseen HT detection tasks. We observe that our proposed method outperforms the TD-Zero method for unseen HT detection tasks across all performance metrics. For instance, our proposed method achieves accuracy and precision values of 0.97 and 1.0, which is 1.05× and 1.12× higher than the TD-Zero method for the same performance metrics. However, our ZSL-based GPT-3.5 Turbo LLM model achieves a competitive accuracy of 92%, closely aligning with TD-Zero's performance (96% accuracy), as shown in Tables 4 and 8.

Furthermore, Figure 5(b) illustrates the comparative performance metrics of our proposed method for HT detection against SOTA ML/DL-based methods in terms of accuracy, FPR, and FNR metrics. We observe that our proposed method performs better than the other SOTA methods. For instance, our proposed method achieves a detection accuracy of 0.97, which is 1.05×, 1.68×, 1.23×, and 1.54× than TD-Zero, SVM, HTNet, and COTD methods. Regarding FPR, our proposed method maintains 0, outperforming TD-Zero, SVM, COTD, and HTNet. The FNR for our proposed method is also the lowest compared to other methods. Overall, our proposed method outperforms other SOTA ML/DL-based methods for unseen HT detection because LLMs can leverage their extensive knowledge base to detect unseen trojans better

TABLE 8. OSL performance in HT detection using different benchmarks using the enterprise GPT-3.5 Turbo and Gemini-1.5 Pro LLM models. The notion X/Y represents the Trust-Hub/ISCAS and accuracy with standard deviation values.

Trojan Status	GPT-3.5 Turbo				Gemini-1.5 Pro			
	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
Trojan-Infected	0.93±0.0/0.99±0.01	0.97/1.0	0.89/0.99	0.91/0.98	0.90±0.01/0.95±0.00	0.93/0.95	0.83/0.92	0.84/0.94
Trojan-Free	0.90±0.02/0.98±0.00	0.85/1.0	0.97/1.0	0.90/1.0	0.86±0.03/0.95±0.01	0.80/0.96	0.97/0.94	0.86/0.95

TABLE 9. OSL performance in HT detection using different benchmarks using the open-source Llama 3.1 and DeepSeek-V2 LLM models. The notion X/Y represents the Trust-Hub/ISCAS and accuracy with standard deviation values.

Trojan Status	Llama 3.1				DeepSeek-V2			
	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
Trojan-Infected	0.81±0.01/0.91±0.00	0.83/0.9	0.76/0.87	0.88/0.98	0.90±0.01/0.94±0.01	0.89/0.93	0.85/0.92	0.91/0.99
Trojan-Free	0.80±0.01/0.89±0.02	0.66/0.88	0.86/0.92	0.79/0.95	0.88±0.01/0.92±0.00	0.80/0.91	0.90/0.94	0.90/0.98

TABLE 10. FSL performance in HT detection using different benchmarks using enterprise GPT-3.5 Turbo and Gemini-1.5 Pro LLM models. The notion X/Y represents the Trust-Hub/ISCAS and accuracy with standard deviation values.

No shots	GPT-3.5 Turbo				Gemini-1.5 Pro			
	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
10	0.82±0.00/0.94±0.00	0.84/0.95	0.95/0.98	0.82/0.96	0.78±0.01/0.87±0.00	0.8/0.9	0.9/0.92	0.79/0.88
12	0.85±0.01/0.98±0.01	0.88/0.99	0.98/1.0	0.87/0.99	0.80±0.02/0.92±0.01	0.81/0.93	0.93/0.95	0.83/0.93
14	0.88±0.03/1.0±0.01	0.89/1.0	1.0/1.0	0.89/1.0	0.84±0.02/0.97±0.02	0.84/0.98	0.96/0.97	0.850.95
16	0.94±0.03/1.0±0.02	0.93/1.0	1.0/1.0	0.92/1.0	0.9±0.01/0.99±0.01	0.89/1.0	0.96/0.99	0.87/0.96

TABLE 11. FSL performance in HT detection using different benchmarks using open-source Llama 3.1 and DeepSeek-V2 LLM models. The notion X/Y represents the Trust-Hub/ISCAS and accuracy with standard deviation values.

No shots	Llama 3.1				DeepSeek-V2			
	Accuracy	Precision	Recall	F1-Score	Accuracy	Precision	Recall	F1-Score
10	0.68±0.04/0.85±0.02	0.69/0.82	0.81/0.86	0.65/0.86	0.72±0.03/0.89±0.01	0.73/0.85	0.84/0.88	0.69/0.89
12	0.74±0.01/0.89±0.03	0.75/0.85	0.84/0.9	0.7/0.88	0.76±0.02/0.91±0.02	0.79/0.87	0.87/0.91	0.76/0.92
14	0.76±0.02/0.91±0.01	0.79/0.86	0.87/0.91	0.73/0.91	0.8±0.03/0.93±0.02	0.81/0.9	0.88/0.94	0.78/0.96
16	0.85±0.01/0.92±0.01	0.83/0.91	0.89/0.93	0.79/0.92	0.88±0.02/0.95±0.00	0.86/0.94	0.92/0.95	0.84/0.96



FIGURE 5. Comparisons with SOTA methods (a) with TD-Zero method, and (b) with other ML/DL-based methods.

than other SOTA MML/DL-based methods, highlighting the efficiency and effectiveness of our methodology in HT detection tasks

Compared to traditional ML/DL-based approaches for HT detection, our LLM-based methodology offers several key advantages. First, our method is *golden-free*, meaning it does not require a reference design for Trojan comparison. This removes a significant barrier to deployment in real-world scenarios where golden chips are often unavailable or impractical to obtain. Second, the training efficiency of our approach is considerably higher. For example, GPT-3.5 Turbo achieves approximately **6× faster fine-tuning** than HTNet, as shown in Table 12, making it highly suitable for rapid development and iteration. Third, our approach exhibits strong *generalization capabilities to unseen Trojans* across heterogeneous circuit families (e.g., Trust-Hub and ISCAS), as demonstrated in our few-shot and one-shot experiments summarized in Tables 8, 9, 11 and 10. Finally, and most importantly, our framework introduces *explainability* into HT detection. Unlike black-box ML/DL models, our LLM-based method produces *human-centric explanations* via Chain-of-Thought prompting. These include specific line numbers, signal names, and trigger mechanisms associated with Trojan behavior, significantly enhancing interpretability and trust, as shown in Figures 6, 8, 7, 9, 10, and 11.

However, we also recognize limitations associated with LLM-based methods. In particular, the enterprise API-based GPT-3.5 Turbo and Gemini-1.5 Pro LLM models introduce a

The Verilog code contains evidence of a Trojan at line numbers **51-54**. The Trojan signal **'trojan_signal'** is manipulated based on a rare condition involving inputs **'g101'** and **'g102'**. This condition acts as a trigger for the Trojan payload, which modifies the output **'g2355'** based on the value of **'trojan_signal'**.

FIGURE 6. Trojan-infected reasoning from ISCAS sample s15850 using enterprise API based GPT-3.5 Turbo LLM model.

The Verilog code provided does not exhibit any evidence of a Trojan. There are no indications of unauthorized data manipulation, hidden triggers, backdoors within the code.

FIGURE 7. Trojan free reasoning from ISCAS sample s15850 using enterprise API based GPT-3.5 Turbo LLM model.

The Verilog code contains a Trojan. The Trojan is located on line 51, where a rare branch-type trigger is introduced. The trigger condition is met when **'g101'** is 1 and **'g102'** is 0. This condition is used to manipulate the output **'g2355'** through a conditional assignment statement on line 59. The trojan signal is stored in the wire **'trojan_signal'**, which is then used to toggle the output **'g2355'** based on its value. This type of Trojan is known as a rare branch Trojan.

FIGURE 8. Trojan-infected reasoning from ISCAS sample s15850 using open source DeepSeek-V2 Turbo LLM model.

The Verilog code provided does not contain any evidence of a Trojan. There are no indications of unauthorized data manipulation, or hidden triggers, within the code.

FIGURE 9. Trojan free reasoning from ISCAS sample s15850 using open source DeepSeek-V2 LLM model.

dependency on external providers, which may raise concerns about long-term model stability, data privacy, or availability. While the open-source DeepSeek-V2 and Llama 3.1 LLM models offer more transparency and local control, they require higher computational resources for fine-tuning compared to lightweight traditional ML models. Additionally, both models are subject to input length limitations, which may necessitate segmentation or truncation strategies when analyzing very large Verilog designs. These limitations are important considerations for deploying LLM-based HT detection at scale and motivate future work in model optimization and circuit partitioning.

F. EVALUATION OF SCALABILITY

Table 12 compares the average overhead of various HT detection methods with our proposed method. We present the required time, memory resources, CPU efficiency during the training and fine-tuning phase, and the necessity for golden chips during testing. The total cost for fine-tuning the enterprise API-based GPT-3.5 Turbo and the

The Verilog code contains a Trojan that manipulates the output of the AES encryption module. The Trojan is triggered by a specific sequence of input states and is designed to leak sensitive information. The Trojan is implemented in the **'TSC'** module (lines 999-1011) where the 'load' wire is manipulated based on the **'Tj_Trig' signal**. The **'Tj_Trig'** signal is generated in the **'Trojan_Trigger'** module (lines 946-978) and is triggered by a sequence of input states that match specific values.

FIGURE 10. Trojan-infected reasoning from Trust-Hub sample AES-T1100 using using enterprise API based GPT-3.5 Turbo LLM model.

The Verilog code contains a Trojan that manipulates the output of the AES encryption module. The Trojan is triggered when the input state matches a specific value (line 17 in the **Trojan_Trigger** module). This is done by the **Trojan_Trigger** module, which sets the **Tj_Trig** wire to 1 when the input state is equal to **128'h00112233_44556677_8899aabb_ccddeeff**. The **Tj_Trig** wire is then used to trigger the LFSR (Line 15 in the TSC module).

FIGURE 11. Trojan-infected reasoning from Trust-Hub sample AES-T1100 using open source DeepSeek-V2 LLM model.

Gemini-1.5 Pro LLM models is $100 and $120 (since we are using the OpenAI and Google-provided API for fine-tuning) on the Trust-Hub, and ISCAS' 85/'89 datasets. Additionally, we account for the token-based pricing models of both API-based GPT-3.5 Turbo and Gemini-1.5 Pro models. The GPT-3.5 Turbo is priced at $0.0015 per 1,000 input tokens and $0.0020 per 1,000 output tokens. On the other hand, the Gemini 1.5 Pro charges $1.25 per million input tokens (up to 128k tokens) and $2.50 per million input tokens (above 128k), with output tokens priced at $5.00 and $10.00 per million tokens, respectively. On average, a single sample from the dataset (e.g., Trust-Hub, and ISCAS' 85/'89 contains 1,200–1,500 tokens per inference (including both input and output tokens) in the ZSL setting. During inference, processing all 400 samples costs approximately $1.45 using GPT-3.5 Turbo, and around $0.80 using Gemini 1.5 Pro for HT detection tasks. In FSL and OSL settings, where prompts include up to 16 examples for FSL and one example for OSL, the total number of tokens per inference increase to around 10,000–15,000 tokens. This results in an total cost of around $20.00 for GPT-3.5 Turbo and $12.40 for Gemini 1.5 Pro to process all 400 samples during the inference phase for HT detection tasks.

However, no additional cost is required for our open-source DeepSeek-V2 and Llama 3.1 LLM models in HT detection tasks. From Table 12, it is observed that our proposed method using the GPT-3.5 Turbo LLM model required only 1680.2 seconds to fine-tune for HT detection, which is 1.09×, 5.92×, 6.4×, 6.9×, 26.32×, 5.94×, and 3.65× lower than our enterprise Gemini model, open-source DeepSeek-V2 model, open-source Llama 3.1 model, TD-Zero, HTNet,

TABLE 12. Comparison of training and fine-tuning cost.

Methods	Time (s)	Memory (MB)	Golden-Chips
SVM	6133.1	67.8	Yes
COTD	9986.7	128.5	No
HTNet	44215	1024.0	No
TD-Zero	11377.6	1377.2	No
Our GPT-3.5 Turbo	1680.2	632.1	No
Our Gemini-1.5 Pro	1839.5	817.4	No
Our Llama 3.1	10800	945.1	No
Our DeepSeek-V2	9950	879.2	No

COTD, and SVM methods. It is important to note that the open-source DeepSeek-V2 and Llama 3.1 models require more time for fine-tuning than the enterprise API-enabled GPT-3.5 Turbo model. This is because it necessitates training the model weights from scratch. Also, GPT-3.5 Turbo and Gemini-1.5 Pro models inference via API averages 0.5–1 second per sample, while DeepSeek-V2 and Llama 3.1's local inference takes 1–2 seconds on our GPU. However, DeepSeek-V2 and Llama 3.1 models provide a cost-effective alternative without dependency on paid services, allowing organizations to utilize existing resources without ongoing financial commitments. Overall, our proposed method (e.g., GPT-3.5 Turbo and Gemini-1.5 Pro LLM models) requires less memory space during fine-tuning time because it does not need to train the model from scratch while providing higher HT detection accuracy.

G. PRIVACY CONSIDERATIONS

While the GPT-3.5 Turbo and Gemini 1.5 Pro models offer notable advantages in terms of training efficiency, ease of integration, and superior detection performance, their use via an enterprise API introduces potential privacy concerns. Specifically, deploying these models may require transmitting proprietary RTL designs to an external cloud-based service, which could raise confidentiality issues in industrial or security-sensitive contexts. Although OpenAI and Google DeepMind provide commercial terms and data usage assurances, such deployment may not meet the data governance requirements of all organizations. In contrast, despite its higher training overhead, the open-source Llama 3.1 and DeepSeek-V2 models can be fully deployed and fine-tuned on local infrastructure, enabling private, offline inference without exposing RTL data to third-party services. This presents a clear trade-off between performance and privacy, and practitioners must weigh these factors when choosing between enterprise and open-source LLM solutions for hardware Trojan detection.

H. TROJAN EXPLANATION

Figures 6, 8, 7, 9, 10, and 11 illustrate how our enterprise GPT-3.5 Turbo and open-source DeepSeek-V2 LLM models show the explanation/reasoning of the Trojan-infected and Trojan-free classification from ISCAS and Trust-Hub datasets for the s15850 and AES-T1100 benchmarks. We focus our explanation/reasoning generation on enterprise GPT-3.5 Turbo and open-source DeepSeek-V2 model, since the enterprise GPT-3.5 Turbo model outperforms the enterprise Gemini 1.5 Pro model in HT detection tasks, and the open-source DeepSeek-V2 model performs better than the open-source Llama 3.1 model, as shown in Tables 4, 5, 6 7, 8, 9, 11 and 10. Specifically, we present only the reasoning outputs from these two models when explaining the classification of detected HTs. This decision ensures that the reported explanation performance reflects the most accurate and interpretable outputs across enterprise-grade and open-source LLMs used in our study. From Figure 6, using the GPT-3.5 Turbo LLM model and s15850 benchmark from ISCAS, it is observed that for the Trojan-infected explanation, the model provides the accurate line numbers where Trojan is injected and the corresponding inputs (i.e., **'g101'** and **'g102'**) as important indicators of Trojan injection. However, no Trojan trigger indication is found for the Trojan-free explanation, as shown in Figure 7.

On the other hand, using the GPT-3.5 Turbo LLM model and AES-T1100 benchmark from Trust-Hub, it is observed that for the Trojan-infected explanation, the model also provides the accurate line numbers where Trojan is injected and the corresponding inputs (i.e., **'TSC'** as important indicators of Trojan injection. A similar phenomenon is also observed for the open-source DeepSeek-V2 model in explanation generation for the same samples. However, one using the open-source DeepSeek-V2 model and AES-T1100 benchmark from Trust-Hub, it is observed that for the Trojan-infected explanation, the model provides the wrong line numbers (i.e., line 17 in the **Trojan_Trigger**) where Trojan is injected, as shown in Figure 11. The reason is that the open-source DeepSeek-V2 LLM model has limited HT detection results (low accuracy) as compared to the enterprise API-based GPT-3.5 Turbo LLM model. This human-centric explanation capability is a significant advancement, as it provides granular insights into the models' classification decisions and reinforces the transparency and trustworthiness of automated HT detection. By explicitly identifying suspicious line numbers and input signals, these human-centric explanations provided by LLM models enhance the interpretability of Trojan detection processes, making them more accessible and reliable for stakeholders in the IC supply chain. This human-centric reasoning supports accountability and trust, which are critical in mitigating risks associated with hardware security-sensitive environments.

1) FAILURE CASE ANALYSIS

While our proposed LLM-based approach demonstrates strong overall performance, we observed several failure cases, particularly with the DeepSeek-V2 model under zero-shot and one-shot settings. These failures include missed Trojan detections in complex designs such as `s15850` and `s35932`, and incorrect reasoning output where the model inaccurately localizes trigger logic or fails to provide a coherent explanation. For instance, in Figure 11, the DeepSeek-V2 model incorrectly identifies line 17 as the

trigger location in the AES-T1100 benchmark. These limitations are likely due to weaker generalization under limited context or fine-tuning, and sensitivity to subtle changes in prompt structure. Addressing these failure modes using prompt engineering, enhanced supervision, or hybrid techniques is a promising direction for future work.

I. APPLICATION OF THE PROPOSED METHOD BEYOND HT DETECTION

The proposed HT detection method leverages pre-trained LLM, adaptable across various domains. For instance, the proposed method, LLMs, enhances intrusion detection systems by analyzing network traffic to identify anomalies indicative of potential security breaches, thereby improving the accuracy and responsiveness of these systems. Furthermore, LLMs can automate the classification and understanding of malicious code in malware analysis, facilitating quicker mitigation strategies. In hardware design and testing, LLMs can contribute to design automation [63], [64], automating design verification processes [65], ensuring that integrated circuits function as intended without vulnerabilities. Furthermore, the proposed HT detection method leverages pre-trained LLMs to scale across larger datasets effectively. By utilizing ICL mechanisms, the model can adapt to new data without extensive retraining, thereby conserving computational resources. In addition, ICL allows the LLM to interpret and analyze hardware designs by incorporating relevant examples directly into the input prompts, enabling the model to provide accurate assessments based on the context provided. This approach is particularly advantageous for handling extensive and complex hardware datasets, as it mitigates the challenges associated with traditional ML/DL-based methods, which require training from scratch. Moreover, the flexibility of ICL is particularly advantageous in handling the increasing complexity of modern ICs, as it allows the model to interpret and analyze new patterns and anomalies within the hardware without the need for exhaustive reconfiguration.

VI. CONCLUSION AND FUTURE WORKS

In this work, we proposed an innovative HT detection and explanation method by leveraging the knowledge of pre-trained LLM models without training a model from scratch. Specifically, we proposed an ICL-based three-learning strategy (i.e., ZSL, FSL, and OSL) for the pre-trained LLMs, namely enterprise API-enabled LLM models (e.g.,GPT-3.5 Turbo and Gemini-1.5 Pro) and open-source LLM models (e.g., DeepSeek-V2 and Llama-3.1)for the HT detection and explanation tasks. We validate our proposed method using diverse benchmarks from Trust-Hub and ISCAS datasets. Our experimental results show that the proposed FSL-based enterprise GPT-3.5 Turbo LLM model and open-source DeepSeek-V2 LLM model detect unknown HTs with an accuracy of 97% and 91% and drastically reduces the training time, which outperforms state-of-the-art other works for these benchmarks in terms of HT detection accuracy and training time. Furthermore, after detecting the HT, it provides the human-centric reasoning/explanation of the detected HT, which provides trustworthiness and insight into why and how the model arrived at a specific decision (e.g., Trojan-infected) and reinforces transparency and trust in the IC supply chain. Thus, in the future, we plan to extend the approach to incorporate more diverse datasets and hardware scenarios, explore enhancing model robustness, and improve explanation quality for greater transparency and trust in HT detection. Additionally, we aim to broaden the applicability of our proposed method by applying it to more complex and modern large scale hardware designs, such as RISC-V, MIPS, neural network accelerators, etc.,. Therefore, we plan to incorporate profiling and segmentation mechanisms to support adaptive prompt generation and memory-efficient inference. These improvements would enable our approach to remain applicable to very large hardware designs without compromising LLM performance or explainability. Furthermore, evaluating our proposed method against more sophisticated forms of obfuscations, such as control flow flattening, logic locking, or randomized register insertion, is a valuable direction, and we plan to incorporate these in future iterations to assess robustness more comprehensively. Moreover, incorporating parameter-efficient fine-tuning techniques such as LoRA and QLoRA-style adapters into the architecture of open-source LLMs (e.g., Llama 3.1 and DeepSeek-V2) could substantially reduce the number of trainable parameters and memory overhead, thereby enhancing the feasibility of deploying these models on edge or resource-constrained devices. Therefore, in future work, we plan to integrate these techniques into our framework to further strengthen the practicality and appeal of open-source LLMs for hardware security applications.

REFERENCES

[1] Z. Pan and P. Mishra, "TD-zero: Automatic golden-free hardware trojan detection using zero-shot learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 7, pp. 1998–2011, Jul. 2024.

[2] K. I. Gubbi, B. S. Latibari, A. Srikanth, T. Sheaves, S. A. Beheshti-Shirazi, S. Rafatirad, A. Sasan, H. Homayoun, and S. Salehi, "Hardware trojan detection using machine learning: A tutorial," *ACM Trans. Embedded Comput. Syst.*, vol. 22, no. 3, pp. 1–26, May 2023.

[3] R. Sharma and P. Ranjan, "Machine learning–based hardware trojans detection in integrated circuits: A systematic review," in *Proc. Int. Conf. Data Sci. Appl.* Springer, 2023, pp. 35–48.

[4] S. Moustakidis, K. Liakos, G. Georgakilas, N. Sketopoulos, S. Seimoglou, P. Karlsson, and F. Plessas, "A novel holistic approach for hardware trojan detection powered by deep learning (hero)," *Attract'20*, 2020.

[5] K. Hasegawa, M. Yanagisawa, and N. Togawa, "A hardware-trojan classification method using machine learning at gate-level netlists based on trojan features," *IEICE Trans. Fundamentals Electron., Commun. Comput. Sci.*, vol. 100, no. 7, pp. 1427–1438, 2017.

[6] S. Faezi, R. Yasaei, and M. A. Al Faruque, "HTnet: Transfer learning for golden chip-free hardware trojan detection," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1484–1489.

[7] H. Salmani, "COTD: Reference-free hardware trojan detection and recovery based on controllability and observability in gate-level netlist," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 2, pp. 338–350, Feb. 2017.

[8] Z. Pan and P. Mishra, "Hardware trojan detection using Shapley ensemble boosting," in *Proc. 28th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2023, pp. 1–8.

[9] J. Ye, X. Chen, N. Xu, C. Zu, Z. Shao, S. Liu, Y. Cui, Z. Zhou, C. Gong, Y. Shen, J. Zhou, S. Chen, T. Gui, Q. Zhang, and X. Huang, "A comprehensive capability analysis of GPT-3 and GPT-3.5 series models," 2023, *arXiv:2303.10420*.

[10] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, and S. Anadkat, "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[11] J. Chaudhuri, D. Thapar, A. Chaudhuri, F. Firouzi, and K. Chakrabarty, "SPICED+: Syntactical bug pattern identification and correction of trojans in A/MS circuits using LLM-enhanced detection," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 33, no. 4, pp. 1118–1131, Apr. 2025.

[12] V. T. Hayashi and W. V. Ruggiero, "Hardware trojan detection in open-source hardware designs using machine learning," *IEEE Access*, vol. 13, pp. 37771–37788, 2025.

[13] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "TrojanWhisper: Evaluating pre-trained LLMs to detect and localize hardware trojans," 2024, *arXiv:2412.07636*.

[14] G. Team, P. Georgiev, and V. I. Lei, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," 2024, *arXiv:2403.05530*.

[15] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, and A. Fan, "The llama 3 herd of models," 2024, *arXiv:2407.21783*.

[16] DeepSeek-AI, "DeepSeek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024, *arXiv:2405.04434*.

[17] *Trusthub.org: Trust-Hub*. Accessed: Jun. 23, 2024. [Online]. Available: https://trust-hub.org/#/home

[18] *ISCAS'*85&89. Accessed: Jun. 23, 2024. [Online]. Available: https://sportlab.usc.edu/ msabrishami/benchmarks.html

[19] A. Nahiyan, M. Sadi, R. Vittal, G. Contreras, D. Forte, and M. Tehranipoor, "Hardware trojan detection through information flow security verification," in *Proc. IEEE Int. Test Conf. (ITC)*, Oct. 2017, pp. 1–10.

[20] Y. Zhao, G. Qu, Q. Zhang, Y. Li, Z. Li, and J. He, "Static gate-level information flow for hardware information security with bounded model checking," in *Proc. IEEE 42nd VLSI Test Symp. (VTS)*, Apr. 2024, pp. 1–7.

[21] M. Tehranipoor, N. N. Anandakumar, and F. Farahmandi, "Security verification," in *Hardware Security Training, Hands-on!*. Springer, 2023, pp. 115–135.

[22] R. S. Chakraborty, F. Wolff, S. Paul, C. Papachristou, and S. Bhunia, "MERO: A statistical approach for hardware trojan detection," in *Proc. 11th Int. Workshop Cryptograph. Hardw. Embedded Syst.*, Jan. 2009, pp. 396–410.

[23] A. Jayasena and P. Mishra, "Directed test generation for hardware validation: A survey," *ACM Comput. Surveys*, vol. 56, no. 5, pp. 1–36, May 2024.

[24] M. A. Nourian, M. Fazeli, and D. Hely, "Hardware trojan detection using an advised genetic algorithm based logic testing," *J. Electron. Test.*, vol. 34, no. 4, pp. 461–470, Aug. 2018.

[25] R. Mukherjee, S. R. Rajendran, and R. S. Chakraborty, "A comprehensive survey of physical and logic testing techniques for hardware trojan detection and prevention," *J. Cryptograph. Eng.*, vol. 12, no. 4, pp. 495–522, Nov. 2022.

[26] Y. Lyu and P. Mishra, "MaxSense: Side-channel sensitivity maximization for trojan detection using statistical test patterns," *ACM Trans. Design Autom. Electron. Syst.*, vol. 26, no. 3, pp. 1–21, May 2021.

[27] A. Jayasena and P. Mishra, "Scalable detection of hardware trojans using ATPG-based activation of rare events," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 12, pp. 4450–4462, Dec. 2023.

[28] S. Narasimhan, D. Du, R. S. Chakraborty, S. Paul, F. G. Wolff, C. A. Papachristou, K. Roy, and S. Bhunia, "Hardware trojan detection by multiple-parameter side-channel analysis," *IEEE Trans. Comput.*, vol. 62, no. 11, pp. 2183–2195, Nov. 2013.

[29] R. Yasaei, S.-Y. Yu, and M. A. Al Faruque, "GNN4TJ: Graph neural networks for hardware trojan detection at register transfer level," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1504–1509.

[30] R. Vishwakarma and A. Rezaei, "Risk-aware and explainable framework for ensuring guaranteed coverage in evolving hardware trojan detection," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 01–09.

[31] Z. Jiang and Q. Ding, "A framework for hardware trojan detection based on contrastive learning," *Sci. Rep.*, vol. 14, no. 1, p. 30847, Dec. 2024.

[32] R. Vishwakarma and A. Rezaei, "Uncertainty-aware hardware trojan detection using multimodal deep learning," in *Proc.Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–6.

[33] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Unleashing GHOST: An LLM-powered framework for automated hardware trojan design," 2024, *arXiv:2412.02816*.

[34] G.-W. Wan, S.-Z. Wong, D. Liu, and X. Wang, "Assessment of pre-trained large language models for hardware trojan detection in RTL designs," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, p. 1.

[35] K. Hasegawa, M. Yanagisawa, and N. Togawa, "Trojan-feature extraction at gate-level netlists and its application to hardware-trojan detection using random forest classifier," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2017, pp. 1–4.

[36] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag, "TabLLM: Few-shot classification of tabular data with large language models," in *Proc. Int. Conf. Artif. Intell. Statist.*, Jan. 2023, pp. 5549–5581.

[37] D. Saha, K. Yahyaei, S. Kumar Saha, M. Tehranipoor, and F. Farahmandi, "Empowering hardware security with LLM: The development of a vulnerable hardware database," in *Proc. IEEE Int. Symp. Hardw. Oriented Secur. Trust (HOST)*, May 2024, pp. 233–243.

[38] G. Kokolakis, A. Moschos, and A. D. Keromytis, "Harnessing the power of general-purpose LLMs in hardware trojan design," in *Proc. Appl. Cryptography Netw. Secur. Workshops*. Cham, Switzerland: Springer Nature, Jan. 2024, pp. 176–194.

[39] J. Bhandari, R. Sadhukhan, P. Krishnamurthy, F. Khorrami, and R. Karri, "SENTAUR: Security EnhaNced trojan assessment using LLMs against undesirable revisions," 2024, *arXiv:2407.12352*.

[40] D. Saha, S. Tarek, K. Yahyaei, S. K. Saha, J. Zhou, M. Tehranipoor, and F. Farahmandi, "LLM for SoC security: A paradigm shift," *IEEE Access*, vol. 12, pp. 155498–155521, 2024.

[41] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.

[42] F. J. Dorfner, L. Jürgensen, L. Donle, F. Al Mohamad, T. R. Bodenmann, M. C. Cleveland, F. Busch, L. C. Adams, J. Sato, T. Schultz, A. E. Kim, J. Merkow, K. K. Bressem, and C. P. Bridge, "Is open-source there yet? A comparative study on commercial and open-source LLMs in their ability to label chest X-ray reports," 2024, *arXiv:2402.12298*.

[43] S. Kukreja, T. Kumar, A. Purohit, A. Dasgupta, and D. Guha, "A literature survey on open source large language models," in *Proc. 7th Int. Conf. Comput. Manage. Bus.*, Jan. 2024, pp. 133–143.

[44] B. Atil, A. Chittams, L. Fu, F. Ture, L. Xu, and B. Baldwin, "LLM stability: A detailed analysis with some surprises," 2024, *arXiv:2408.04667*.

[45] A. Webson and E. Pavlick, "Do prompt-based models really understand the meaning of their prompts?" 2021, *arXiv:2109.01247*.

[46] J. Lee, X. Wang, D. Schuurmans, M. Bosma, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, Jan. 2022, pp. 24824–24837.

[47] Z. Yu, L. He, Z. Wu, X. Dai, and J. Chen, "Towards better chain-of-thought prompting strategies: A survey," 2023, *arXiv:2310.04959*.

[48] M. Zanella and I. B. Ayed, "Low-rank few-shot adaptation of vision-language models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 1593–1603.

[49] OpenAI. (2023). *GPT-3.5 Turbo Fine-Tuning and API Updates*. [Online]. Available: https://openai.com/blog/gpt-3-5-turbo-fine-tuning-and-api-updates

[50] B. Manibalan, J. A. A. Jothi, and V. M. A. Rajam, "CommandSense: Actions-objects-results prediction from natural language commands using fine-tuned GPT-3.5 turbo," in *Proc. IEEE 3rd World Conf. Appl. Intell. Comput. (AIC)*, Jul. 2024, pp. 75–80.

[51] Z. Qian, X. Xu, and X. Zhang, "ZXQ at SemEval-2024 task 7: Fine-tuning GPT-3.5-turbo for numerical reasoning," in *Proc. 18th Int. Workshop Semantic Eval. (SemEval)*, A. K. Ojha, A. S. Doğruöz, H. T. Madabushi, G. D. S. Martino, S. Rosenthal, and A. Rosá, Eds., Mexico: Association for Computational Linguistics, 2024, pp. 218–223. [Online]. Available: https://aclanthology.org/2024.semeval-1.34/

[52] E. Latif and X. Zhai, "Fine-tuning ChatGPT for automatic scoring," *Comput. Education: Artif. Intell.*, vol. 6, Feb. 2024, Art. no. 100210.

[53] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019, *arXiv:1711.05101*.

[54] H. Touvron and L. Martin, "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.

[55] V. Gohil, H. Guo, S. Patnaik, and J. Rajendran, "ATTRITION: Attacking static hardware trojan detection techniques using reinforcement learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 1275–1289.
[56] Z. Pan and P. Mishra, "A survey on hardware vulnerability analysis using machine learning," *IEEE Access*, vol. 10, pp. 49508–49527, 2022.
[57] K. Inaba, T. Yoneda, T. Kanamoto, A. Kurokawa, and M. Imai, "Hardware trojan insertion and detection in asynchronous circuits," in *Proc. 25th IEEE Int. Symp. Asynchronous Circuits Syst. (ASYNC)*, May 2019, pp. 134–143.
[58] Y. Li, H. Li, Z. Pu, J. Zhang, X. Zhang, T. Ji, L. Sun, C. Li, and H. Chen, "Is large language model good at database knob tuning? A comprehensive experimental evaluation," 2024, *arXiv:2408.02213*.
[59] *Hugging Face*. Accessed: Nov. 23, 2024. [Online]. Available: https://huggingface.co/meta-llama/Llama-3.1-8B
[60] *VLLM*. Accessed: Nov. 23, 2024. [Online]. Available: https://github.com/vllm-project/vllm
[61] *Llama 3.1–405b, 70b & 8b With Multilinguality and Long Context*. Accessed: Dec. 4, 2024. [Online]. Available: https://huggingface.co/blog/llama31
[62] M. Xia, T. Gao, Z. Zeng, and D. Chen, "Sheared LLaMA: Accelerating language model pre-training via structured pruning," 2023, *arXiv:2310.06694*.
[63] R. Ma, Y. Yang, Z. Liu, J. Zhang, M. Li, J. Huang, and G. Luo, "VerilogReader: LLM-aided hardware test generation," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, Jun. 2024, pp. 1–5.
[64] C. Xiao, Y. Deng, Z. Yang, R. Chen, H. Wang, J. Zhao, H. Dai, L. Wang, Y. Tang, and W. Xu, "LLM-based processor verification: A case study for neuromorphic processor," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2024, pp. 1–6.
[65] K. Xu, R. Qiu, Z. Zhao, G. Li Zhang, U. Schlichtmann, and B. Li, "LLM-aided efficient hardware design automation," 2024, *arXiv:2410.18582*.



RIPAN KUMAR KUNDU (Student Member, IEEE) received the B.Sc. degree in electrical and electronic engineering from American International University Bangladesh (AIUB), Dhaka, Bangladesh, in 2016, and the M.S. degree in electrical engineering from the University of Rostock, Rostock, Germany, in 2022. He is currently pursuing the Ph.D. degree in electrical and computer engineering. He is a Graduate Research Assistant with the Dependable Cyber-Physical Systems (DCPS) Laboratory, University of Missouri, Columbia, MO, USA. His research interests include cyber and hardware security, trustworthy AI/ML, and explainable and fair ML.



KHURRAM KHALIL (Student Member, IEEE) received the B.S. degree in electronic engineering and the M.S. degree in robotics and artificial intelligence from the School of Mechanical and Manufacturing Engineering (SMME), National University of Sciences and Technology (NUST), Pakistan, in 2021. He is currently pursuing the Ph.D. degree in computer science. He is a Graduate Research Assistant with the Dependable Cyber-Physical Systems (DCPS) Laboratory, University of Missouri, Columbia, MO, USA. His research interests include optimization techniques for generative models, AI security, and the development of robust AI systems.



ERIC GARCIA is currently pursuing the B.S. degree in computer science with an emphasis in artificial intelligence and a minor in mathematics with Columbia College, Columbia, MO, USA. His research interests include cybersecurity and artificial intelligence.



ETHAN GRASSIA is currently pursuing the B.S. degree in computer science with an emphasis in cyber security and a minor in mathematics with Loyola University, Chicago, IL, USA. His research interests include embedded hardware, artificial intelligence, and cybersecurity.



PRASAD CALYAM (Senior Member, IEEE) received the M.S. and Ph.D. degrees from the Department of Electrical and Computer Engineering, The Ohio State University, in 2002 and 2007, respectively. He is currently the Greg L. Gilliom Professor of cybersecurity with the Department of Electrical Engineering and Computer Science, University of Missouri, Columbia, MO, USA. Previously, he was the Research Director of the Ohio Supercomputer Center. His current research interests include distributed and cloud computing, computer networking, and cybersecurity.



KHAZA ANUARUL HOQUE (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees from the Department of Electrical and Computer Engineering, Concordia University, Montreal, Canada, in 2011 and 2016, respectively. He is currently an Associate Professor with the Department of Electrical Engineering and Computer Science, University of Missouri, Columbia, MO, USA, where he directs the Dependable Cyber-Physical Systems (DCPS) Laboratory. Before joining MU, he was an FRQNT Postdoctoral Fellow with the University of Oxford, U.K. His research interests include AI hardware, design automation, cyber-physical systems, and safe AI/ML. He is a Senior Member of ACM and a member of AAAS and the IEEE Technical Committee on Cyber-Physical Systems (CPS). He has received several awards and distinctions, including the 2023 IEEE/ACM MEMOCODE Best Paper Nomination, the FQRNT Postdoctoral Fellowship Award, in 2016, the FQRNT Doctoral Research Award, in 2012, and the Best Paper Award at the 8th IEEE International NEWCAS Conference, in 2011. He is an Associate Editor of *ACM Transactions on Probabilistic Machine Learning* (TOPML) journal.